Complete programs for the N005 application

Human Compression Project

The first program is required only for the three small affine parameters $s = 2, 3, 4$. The other two programs validate presentation conventions and the literal $s = 1$ twist. All three complete files are included. Their roles do not change because they share an appendix.

1 Required finite affine verification

```
#!/usr/bin/env python3
"""Exact integer/parabolic certificate for source Example 5.11; no GAP."""
from itertools import combinations
from functools import lru_cache
import json
IDENTITY=(tuple(range(4)),(0,0,0,0))
def need(ok,message):
    if not ok: raise RuntimeError(message)
def mul(a,b):
    p,v=a;q,w=b
    return tuple(p[q[i]] for i in range(4)),tuple(v[i]+w[p.index(i)] for i in range(4))
def refl(i,j,v=(0,0,0,0)):
    p=list(range(4));p[i],p[j]=p[j],p[i]
    return tuple(p),v
MARKS=(refl(0,1),refl(1,2),refl(0,3,(1,0,0,-1)),refl(2,3))
def word(seq):
    out=IDENTITY
    for i in seq:out=mul(out,MARKS[i])
    return out

def subgroup(gens,operation=mul,unit=IDENTITY,limit=100000):
    found={unit};todo=[unit]
    while todo:
        a=todo.pop()
        for b in gens:
            c=operation(a,b)
            if c not in found:
                found.add(c);todo.append(c)
                need(len(found)<=limit,'subgroup cap')
    return frozenset(found)

def proper():
    return [subgroup([MARKS[i] for i in range(4) if mask>>i&1],limit=25) for mask in range(15)]

def model(s):
    modulus=2*s
    def canon(v):
        a=tuple(x%modulus for x in v)
        b=tuple((x+s)%modulus for x in v)
        return min(a,b)
    def image(a):return a[0],canon(a[1])
    def op(a,b):return image(mul(a,b))
    return image,op

def validate():
    need(all(mul(a,a)==IDENTITY for a in MARKS),'involutions')
    for i,j in combinations(range(4),2):
        exponent=2 if {i,j} in ({0,3},{1,2}) else 3
        need(word([i,j]*exponent)==IDENTITY,'Coxeter relation')
    translation=word([2,3,0,1]*2)
    need(translation==(tuple(range(4)),(1,1,-1,-1)),'translation word')
    P=proper()
    need([len(P[m]) for m in [7,11,13,14]]==[24]*4,'four finite parabolics')
```

```
    need(max(abs(v) for H in P for _,t in H for v in t)==1,'coordinate bound')
    products={(i,j):frozenset(mul(a,b) for a in P[i] for b in P[j]) for i in range(15) for j in
    range(15)}
    need(max(abs(v) for H in products.values() for _,t in H for v in t)==2,'product coordinate bound')
    rows=[]
    for s in [1,2,3,4]:
        image,op=model(s)
        Q=[frozenset(map(image,H)) for H in P]
        product={(i,j):frozenset(map(image,H)) for (i,j),H in products.items()}
        ip=[]
        for i in range(15):
            for j in range(15):
                if Q[i]&Q[j]!=Q[i&j]:ip.append([i,j,len(Q[i]&Q[j]),len(Q[i&j])])
        failure=[]
        if not ip:
            for i in range(15):
                for j in range(15):
                    for k in range(15):
                        if product[i,j]&product[i,k]!=product[i,j&k]:failure.append([i,j,k])
        if s>=2:
            need(not ip,'positive IP');need(not failure,'positive product identity')
            need(all(len(Q[i])==len(P[i]) for i in range(15)),'proper injection')
        G=subgroup([image(a) for a in MARKS],op,image(IDENTITY))
        need(len(G)==96*s**3,'quotient order')
        row={'s':s,'order':len(G),'generated_parabolic_IP_failures':ip,'product_failures_if_IP':failure,
        'IP_pairs':225,'product_triples':3375 if not ip else 0}
        if s==1:
            maxs=[Q[15^(1<<i)] for i in range(4)]
            intersections={0:G}
            for mask in range(1,16):
                intersections[mask]=frozenset.intersection(*(maxs[i] for i in range(4) if mask>>i&1))
            rc=[]
            for mask,H in intersections.items():
                missing=[i for i in range(4) if not mask>>i&1]
                if len(missing)>=2:
                    childgens=set().union(*(intersections[mask|1<<i] for i in missing))
                    child=subgroup(childgens,op,image(IDENTITY))
                    if child!=H:rc.append({'flag_mask':mask,'parabolic_order':len(H),'generated_children
                    _order':len(child),'witness':sorted(H-child)[0]})
            need(rc,'negative residual connectivity')
            row['residual_generation_failures']=rc
        rows.append(row)
    return {'status':'PASS','translation_word':translation,'integer_proper_orders':[len(x) for x in P],'
    proper_coordinate_bound':1,'product_coordinate_bound':2,'uniform_written_threshold':5,'models':rows}

if __name__=='__main__':print(json.dumps(validate(),sort_keys=True,indent=2))
```

2 Presentation validation

```
#!/usr/bin/env python3
"""Validation of full-space braid polynomials and free-group basis adapters."""
import json
import sympy as sp

def need(ok,msg):
    if not ok:raise RuntimeError(msg)
def word(A,B,m):
    out=sp.eye(A.rows)
    for i in range(m):out=out*(A if i%2==0 else B)
    return out.applyfunc(sp.expand)
def zero(M):return all(sp.simplify(v)==0 for v in M)

def check_reflections():
    z,u,v,r,s=sp.symbols('z u v r s')
    checked=[]
    for m in [2,3,4,5,6,8,10]:
        if m==2:
            A=sp.Matrix([[u*u,0,r],[0,1,0],[0,0,1]])
            B=sp.Matrix([[1,0,0],[0,v*v,s],[0,0,1]])
        elif m%2:
            d=-4*z*sp.cos(sp.pi/m)**2
            A=sp.Matrix([[z,1,r],[0,1,0],[0,0,1]])
```

```
            B=sp.Matrix([[1,0,0],[d,z,s],[0,0,1]])
        else:
            d=2*u*v*sp.cos(2*sp.pi/m)-u*u-v*v
            A=sp.Matrix([[u*u,1,r],[0,1,0],[0,0,1]])
            B=sp.Matrix([[1,0,0],[d,v*v,s],[0,0,1]])
        need(zero(word(A,B,m)-word(B,A,m)),'full-space symbolic braid '+str(m))
        checked.append(m)
    badA=sp.Matrix([[4,1,1],[0,1,0],[0,0,1]])
    badB=sp.Matrix([[1,0,0],[-25,9,2],[0,0,1]])
    need(not zero(badA*badB-badB*badA),'invalid k=1 shortcut must fail')
    return {'full_space_symbolic_labels':checked,'external_coordinate_couplings':'two independent
    symbolic variables','negative_even_label_two_rejected':True}

def reduce(w):
    result=[]
    for a in w:
        if result and result[-1]==-a:result.pop()
        else:result.append(a)
    return tuple(result)
def inv(w):return tuple(-x for x in w[::-1])
def subst(w,images):return reduce([b for x in w for b in (images[x] if x>0 else inv(images[-x]))])

def check_free():
    # Named source order a1,a2,b3,b4,c5,c6,t -> x,y,t.
    x=(1,);y=(2,);t=(3,)
    tx=reduce(inv(t)+x+t);ty=reduce(inv(t)+y+t)
    images=[{1:tx,2:ty,3:x,4:tx,5:y,6:x,7:t},
            {1:x,2:y,3:tx,4:x,5:y,6:x,7:t},
            {1:x,2:y,3:x,4:tx,5:y,6:x,7:t}]
    relators=[[(1,-4),(3,-6),(-7,5,7,-2),(-7,6,7,-1)],
              [(4,-1),(1,-6),(2,-5),(-7,6,7,-3)],
              [(1,-6),(6,-3),(2,-5),(-7,1,7,-4)]]
    for im,rels in zip(images,relators):
        for rel in rels:need(subst(rel,im)==(),'source relation under F3 map')
    need(ty!=y,'source T1 a2=c5 slip must not be accepted')
    return {'trees':3,'relators_checked':sum(map(len,relators)),'false_T1_equality_rejected':True}

if __name__=='__main__':print(json.dumps({'status':'PASS','reflections':check_reflections(),'free_group_
maps':check_free()},sort_keys=True,indent=2))
```

3 Literal twist validation

```
#!/usr/bin/env python3
"""Independent literal finite semidirect implementation of Example 5.11, s=1."""
from affine_example import MARKS, IDENTITY, model, mul, need, subgroup
from collections import deque
import json

def compose(p,q):return tuple(p[q[i]] for i in range(len(p)))

def check():
    image,operation=model(1);one=image(IDENTITY);marks=tuple(map(image,MARKS))
    words={one:()};queue=deque([one])
    while queue:
        a=queue.popleft()
        for i,g in enumerate(marks):
            b=operation(a,g)
            if b not in words:words[b]=words[a]+(i,);queue.append(b)
    need(len(words)==96,'literal affine quotient size')
    p=(1,0,3,2);q=(0,2,1,3);bid=tuple(range(4))
    outer=subgroup([p,q],compose,bid)
    need(len(outer)==8,'diagram group')
    action={}
    for b in outer:
        transform={}
        for a,w in words.items():
            z=one
            for i in w:z=operation(z,marks[b[i]])
            transform[a]=z
        need(len(set(transform.values()))==96,'automorphism bijection')
        for a in words:
            for i,g in enumerate(marks):
```

```
                need(transform[operation(a,g)]==operation(transform[a],marks[b[i]]),'automorphism
                relator transition')
            action[b]=transform
        for b in outer:
            for c in outer:
                for a in words:
                    need(action[b][action[c][a]]==action[compose(b,c)][a],'action multiplication')
        unit=(one,bid)
        def op(a,b):return operation(a[0],action[a[1]][b[0]]),compose(a[1],b[1])
        x=(marks[0],bid);pp=(one,p);qq=(one,q);gen=(x,pp,qq)
        allg=frozenset((a,b) for a in words for b in outer)
        generated=subgroup(gen,op,unit)
        need(generated==allg,'full generation')
        subs=[subgroup([gen[i] for i in range(3) if mask>>i&1],op,unit) for mask in range(8)]
        need([len(subs[m]) for m in [3,5,6]]==[12,4,8],'rank-two orders')
        pairs=0;triples=0
        for i in range(8):
            for j in range(8):
                need(subs[i]&subs[j]==subs[i&j],'full IP');pairs+=1
        products={(i,j):frozenset(op(a,b) for a in subs[i] for b in subs[j]) for i in range(7) for j in
        range(7)}
        for i in range(7):
            for j in range(7):
                for k in range(7):
                    need(products[i,j]&products[i,k]==products[i,j&k],'coset Helly product');triples+=1
        # Literal Definition4.6: first intersect actual quotient maximal parabolics.
        base_max=[subgroup([marks[j] for j in range(4) if i!=j],operation,one) for i in range(4)]
        need(all(len(h)==24 for h in base_max),'quotient maximal orders')
        one_support=frozenset.intersection(*base_max[1:])
        two_support=base_max[2]&base_max[3]
        borel=frozenset.intersection(*base_max)
        need([len(one_support),len(two_support),len(borel)]==[2,6,1],'actual source intersections')
        source_x=subgroup([(a,bid) for a in one_support]+[qq],op,unit)
        source_y=subgroup([(a,bid) for a in two_support]+[pp],op,unit)
        source_orbit=subgroup([(a,bid) for a in borel]+[pp,qq],op,unit)
        need((source_orbit,source_x,source_y)==(subs[6],subs[5],subs[3]),'literal-to-marked parabolics')
        return
        {'status':'PASS','base_order':96,'outer_order':8,'whole_order':len(allg),'proper_orders':[len(s) for
        s in subs[:7]],'actual_base_intersection_orders':[len(one_support),len(two_support),len(borel)],'IP_
        pairs':pairs,'proper_product_triples':triples,'conclusion':'The literal s=1, F0=1 twist is thin,
        residually connected and flag-transitive; source nonregular inference fails.'}
    if __name__=='__main__':print(json.dumps(check(),indent=2,sort_keys=True))
```